Merge Sort and Divide-and-Conquer

Divide and conquer solves strictly smaller instances and combines their results. State boundaries, base cases, merge preconditions, and results. For $A[p..r]$, length is $m = r - p + 1$, with $p > r$ denoting an empty interval.

```
MERGE-SORT(A,p,r)
    if p >= r: return
    q = p + floor((r-p)/2)
    MERGE-SORT(A,p,q)
    MERGE-SORT(A,q+1,r)
    MERGE(A,p,q,r)
```

A legal nonempty call has $1 \le p \le r \le n$. The full call is $(1, n)$, giving immediate return at $(1, 0)$ for empty input. Child lengths are $\lceil m/2 \rceil$ and $\lfloor m/2 \rfloor$, both smaller for $m \ge 2$. Computing the midpoint from a difference avoids the usual $p + r$ overflow, provided the index range itself is representable.

MERGE without an extra key value

Assume $1 \le p \le q < r \le n$ and two stably sorted halves. L and R are **independent copies**, not views affected by overwriting A. Their lengths are $q - p + 1$ and $r - q$.

```
MERGE(A,p,q,r)
    L = copy(A[p..q]); R = copy(A[q+1..r])
    i = 1; j = 1; k = p
    while i <= L.length and j <= R.length
        if L[i].key <= R[j].key
            A[k] = L[i]; i = i+1
        else
            A[k] = R[j]; j = j+1
        k = k+1
    while i <= L.length
        A[k] = L[i]; i = i+1; k = k+1
    while j <= R.length
        A[k] = R[j]; j = j+1; k = k+1
```

Choose L on equality. Every original left-half record precedes every right-half record, so stability is preserved. Length checks require no key-domain value named infinity.

Proof of merge preservation, order, and termination

Before the main loop,

$$k - p = (i - 1) + (j - 1).$$

The output $A[p..k-1]$ is exactly the stable merge of consumed prefixes $L[1..i-1], R[1..j-1]$. It is sorted, and every emitted key is at most every unconsumed key. Remaining records still exist in independent buffers.

The empty output establishes initialization. When both inputs remain nonempty, their heads are their smallest remaining records. Choosing the smaller head, with left priority on ties, gives the next record of the stable order. Consuming one record and incrementing k preserves lengths, data, order, and stability. Remaining record count decreases by one, proving termination.

At main-loop exit, at least one buffer is exhausted. The other is sorted with keys no smaller than the output tail, so appending it preserves the invariant. Finally $i = |L| + 1, j = |R| + 1, k = r + 1$, and all m records occur once. Without copies, an output write could destroy unread input and invalidate the proof.

Complete merge with duplicate keys

Let $L = [1a, 4b, 4c]$, $R = [2d, 4e, 6f]$, with L originally before R.

i,j before test	Comparison	Output prefix
1,1	$1a \le 2d$	1a
2,1	$4b > 2d$	1a,2d
2,2	$4b \le 4e$	1a,2d,4b
3,2	$4c \le 4e$	1a,2d,4b,4c
4,2	L exhausted; no key comparison	append 4e,6f

There are 4 key comparisons, 6 output writes, and 6 copied buffer records. Interleaving until the last two records can require $m - 1 = 5$ comparisons. Θ(m) work does not mean exactly m key comparisons.

Complete merge with duplicate keys (continued)

Consume one, emit one; output writes cannot corrupt copied inputs

	L			R			
Start	1a	4b	4c	2d	4e	6f	Output: empty
After 1a,2d	—	4b	4c	—	4e	6f	Output: 1a,2d
Take left on ties	—	—	—	—	4e	6f	Output: 1a,2d,4b,4c

L exhausted: append 4e,6f. Equal-key records 4b,4c,4e retain their original order.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Sentinel preconditions

A sentinel implementation can append an infinity record to each buffer and select the smaller head exactly m times, avoiding repeated exhaustion tests. The sentinel key must strictly exceed every legal key, and its record must not be legal input. The maximum legal integer is unsafe as a sentinel.

If legal keys include $+\infty$, take $L = [+\infty]$, $R = [+\infty, +\infty]$. Left priority can select the real left record, then its equal-valued sentinel, then read beyond L on the third output. This is not an out-of-domain marker.

Length checks work for any totally ordered key domain. A tagged sentinel is another option, but it must rank strictly above every real record and never be emitted as data.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

From merge to complete sorting

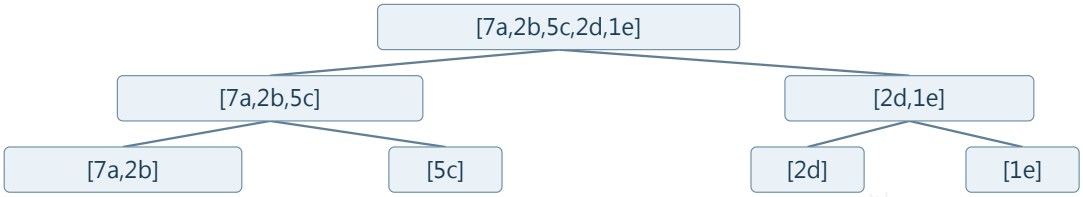
Use strong induction on interval length m. At $m \leq 1$, input is sorted and unchanged. For $m \geq 2$, both smaller calls stably sort and preserve their records by induction. MERGE's preconditions hold, and its proof gives a stable sorting of the whole interval, leaving outside storage unchanged. Child sizes decrease to 0 or 1, proving termination without a power-of-two assumption.

For [7a,2b,5c,2d,1e], splits have lengths 3 and 2, then 2/1 and 1/1. The figure shows every merged node. At the root tie, left 2b still precedes right 2d.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

From merge to complete sorting (continued)

Five records: split downward, return stable sorted results



Bottom up: [2b,7a] → [2b,5c,7a]; right half → [1e,2d]

Root merge: [1e,2b,2d,5c,7a]; key-2 tie: left 2b before right 2d

Child lengths 3 and 2; singletons return. The final split of [7a,2b] into singletons is omitted.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Include merging in the recurrence

Midpoint computation costs Θ(1). Copying and outputting m records costs Θ(m), giving

$$T(m) = T(\lceil m/2 \rceil) + T(\lfloor m/2 \rfloor) + \Theta(m), \quad T(0), T(1) = \Theta(1).$$

For $m = 2^h$, first solve the exact surrogate $U(1) = d$, $U(m) = 2U(m/2) + cm$, with positive c,d. Depth $\ell < h$ has 2^ℓ nodes costing $cm/2^\ell$ each, hence cm per level. There are h internal levels and m leaves of cost d, so $U(m) = cm \lg m + dm$.

Actual internal costs lie between fixed positive multiples of size, and leaves have constant lower and upper costs. Apply the expansion to both bounds to prove Θ(m log m), rather than assuming one exact c describes every instruction and base case.

Arbitrary lengths and a six-record cost table

At depth ℓ, each surviving interval has size at most $\lceil m/2^\ell \rceil$, so height is at most $\lceil \lg m \rceil$. Intervals at one depth are disjoint with total size at most m, proving O(m log m).

At depths $\ell < \lfloor \lg m \rfloor$, each interval has size at least $\lfloor m/2^\ell \rfloor \geq 2$; none has become a leaf. Total size is exactly m, giving Ω(m) at each of $\lfloor \lg m \rfloor$ levels. For $m \geq 2$, this is at least $(\lg m)/2$, proving the lower bound for arbitrary m.

Depth for m=6	Sizes	Total internal size
0	6	6
1	3,3	6
2	2,1,2,1	4
3	1,1,1,1; two earlier leaves	0

Buffer copies and output writes each total 16, not 6 at every depth. There are six base cases.

Peak space differs from cumulative allocation

MERGE allocates L/R only after both recursive calls return and releases them after merging. A length-m merge holds m buffer records. Child buffers do not coexist with parent buffers, giving Θ(n) peak auxiliary record space plus Θ(log n) stack space, hence Θ(n) overall for $n \geq 2$.

Cumulative allocation may be Θ(n log n), but it is not simultaneous demand. Retaining completed buffers changes the lifetime model and invalidates this peak bound. A reusable n-record scratch buffer is also possible with the same nonaliasing requirement.

Insertion sort needs only a saved record and constant index words. This merge sort guarantees Θ(n log n) time but is not in-place. Both are stable under their respective strict-shift and left-tie rules.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study